这一讲，专门讲工具

模型 + 工具 + 循环

1 工具是什么，能让模型多做哪些事？ 上一讲的问题：代码 vs JSON

2 一次工具调用，在底层怎么发生？ 格式怎么保证不出错

3 怎么接给 agent，调得又快又稳？ API 还是 MCP · 怎么评测

工具：模型调用外部程序的接口





课件 p.3（中文重绘）

工具的两种好处：扩展和便利

扩展

让模型做到它根本做不到的事

参数❄冻住

训练结束那一刻

"现在几点？"

不调外部函数 → 不可能知道

便利

模型能做，但外部程序更快、更可靠

```
4817263 × 3952718
= 4817263 ×
3000000
+ 4817263 × 952718
…一步步推
```

计算器

19041282170834



What Are Tools Anyway?（Wang et al., COLM 2024）

什么时候值得调工具

需要	工具	好处
最新信息	搜索	用新鲜的证据回答
精确计算	计算器 · Python	执行可靠
你自己的数据	账户 API	授权范围内的私有状态
改变外部世界	浏览器 · API	真的去做





What Are Tools Anyway?（Wang et al., COLM 2024）

拆解 ChatGPT：从能帮我做什么倒推

聊天

帮我算 18、23、41、54 的平均数

execute_code
(18+23+41+54)/4 = 34

画一个在图书馆学习的机器人

generate_image
watercolor robot studying in a library, warm light, …

帮我建个购物车：香蕉、牛奶、咖啡

create_grocery_cart
购物车 C17 · 3 件

1 `finish(text)` 直接回答 · harness 的控制动作，不是外部程序

2 `search_web(query)` 先检索再回答（RAG）

3 `execute_code(code)` 隔离环境里跑程序

4 `generate_image(query)` 功夫在传什么查询

5 `create_grocery_cart(items)` 你自己定义的函数



课件 p.6–12（中文重绘）

代码是一种元工具

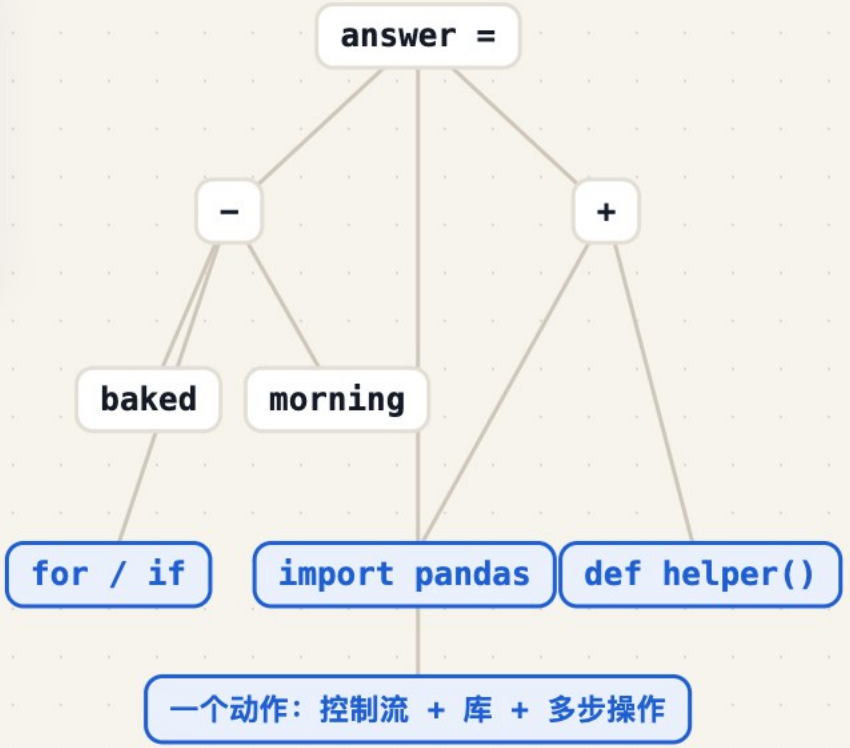
几十个接口的清单

每一步调一个

CodeLM

```
baked, returned = 200, 6
morning, afternoon = 93, 39
answer = baked - morning
  - afternoon + returned
  # → 62
```

面包师烤了 200 个面包，上午卖 93、下午卖 39，退回 6 个，还剩多少？





CodeAct（Wang et al., ICML 2024）原文示例

四个国家，哪里买手机最划算?

一步一个工具（JSON）

美国 ● ● ● ● 汇率 · 价格 · 换算加税 · 运费

日本 ● ● ● ● 汇率 · 价格 · 换算加税 · 运费

德国 ● ● ● ● 汇率 · 价格 · 换算加税 · 运费

印度 ● ● ● ● 汇率 · 价格 · 换算加税 · 运费

来回 16 轮

写成代码（CodeAct）

一次动作

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}
for country in countries:
    rate, tax = lookup_rates(country)
    local = lookup_phone_price("CodeAct 1", country)
    price = convert_and_tax(local, rate, tax)
    ship = estimate_shipping_cost(country)
    final_prices[country] = estimate_final_price(price, ship
min(final_prices, key=final_prices.get)
  # → Japan
```

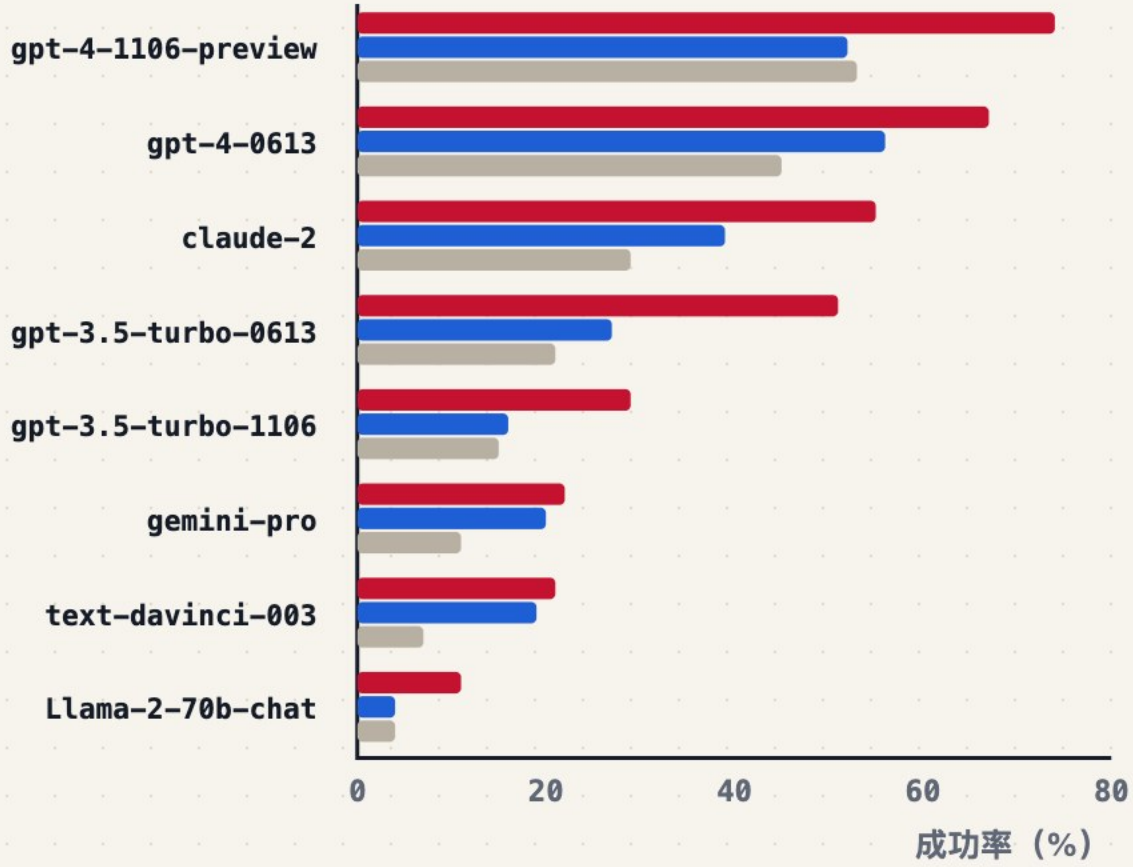


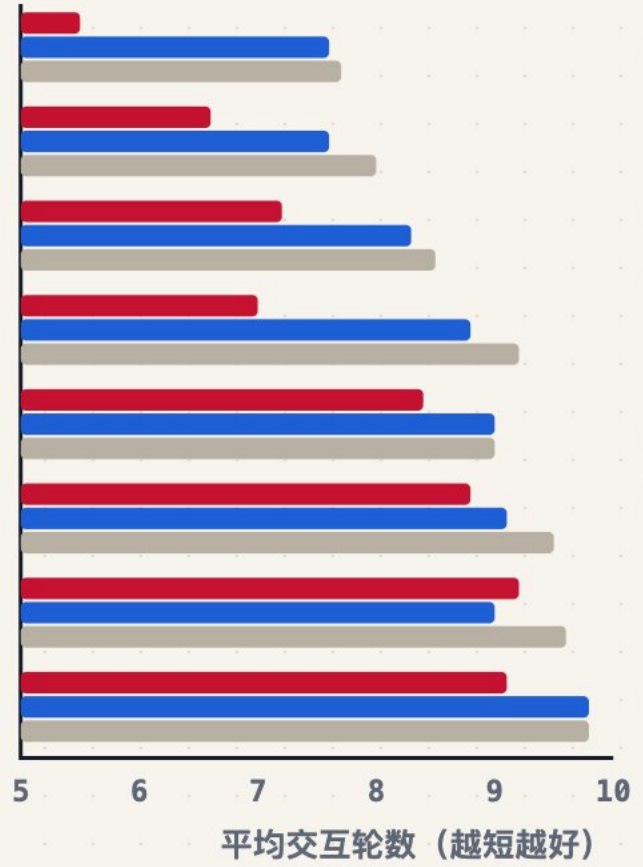
CodeAct（Wang et al., ICML 2024）原文示例

用代码当动作：成功率更高，轮数更少

代码 JSON 文字

这些任务原本只是普通的工具调用任务，不要求写代码

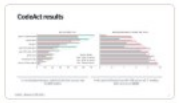




8/8
成功率最高

7/8
轮数最少

12/17
全部 17 个模型，两项都是



CodeAct（Wang et al., ICML 2024）· 按原图读数

代码的代价：权力太大



表达力强

可能写成死循环；引入的库可能被投过毒 → agent 被攻破 → 整个系统被攻破



更难约束

动作空间太宽，没法像带类型的窄函数那样精确校验



影响更大

能动文件、网络和进程，吃掉内存、磁盘和 CPU

沙箱 **资源限制** **权限** **审计日志** → 安全那一讲细讲



课件 p.17（中文重绘）

工具调用，也是 token

普通续写 今天 天气 晴

工具续写 今天 天气 <tool_call> get_weather (city=…)

特殊标记 + 工具名 + 参数

模型：提出动作 → harness：执行



课件 p.19（中文重绘）

一次输出，常常三样都有

思考 ……（推理模型）	"我去查一下天气"	<tool_call>…
不给用户看	给用户看	真正要执行

上一讲的 ReAct：现在就是这样 一次完整的生成



课件 p.19（中文重绘）

特殊标记是什么时候学会的?

预训练时就得在词表里吗? 不一定

预训练

整个互联网

中期训练

已经是目标格式的数据 · 特殊标记在这里加进来

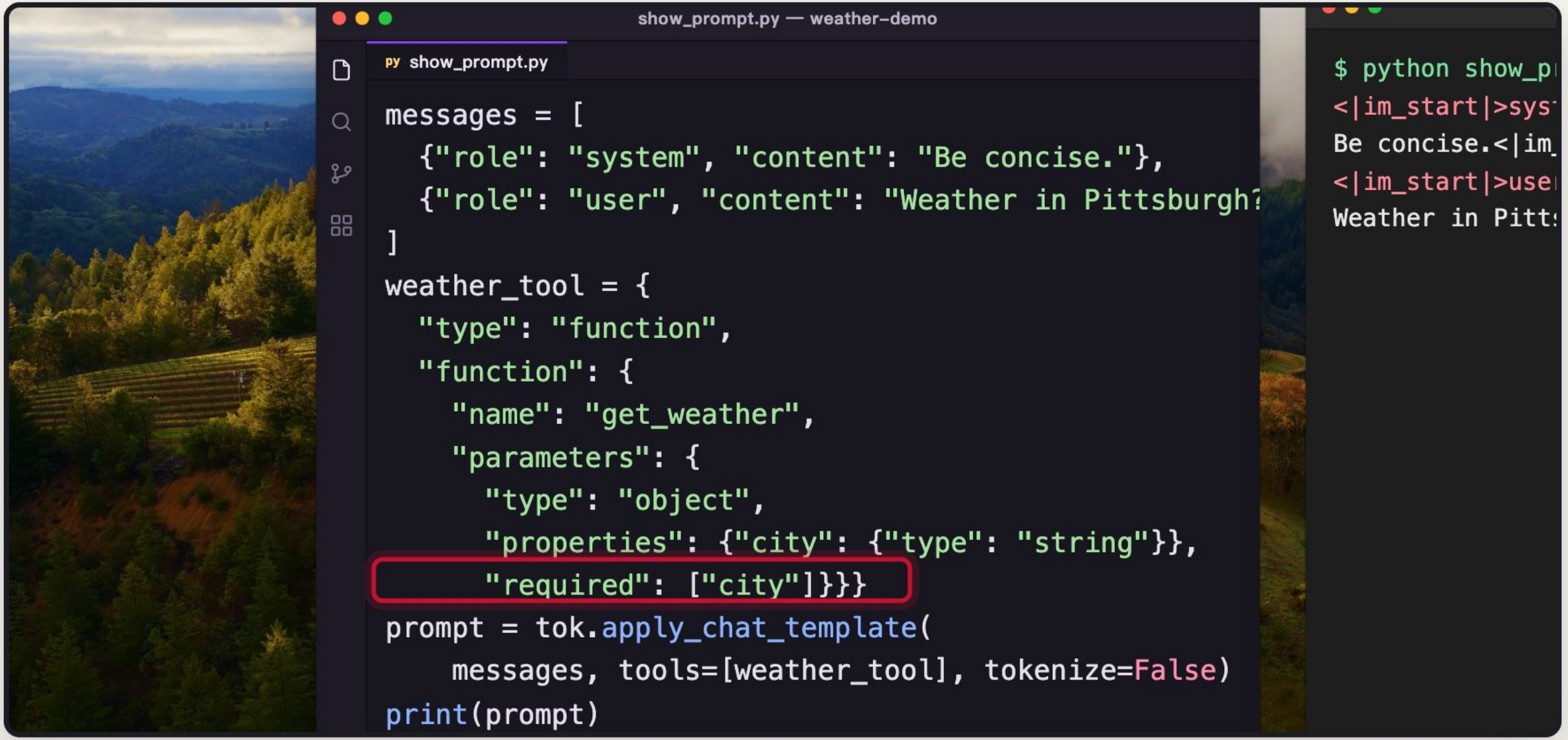
强化学习

之后再做



课件 p.19（中文重绘）

③ 一次工具调用在底层怎么发生





课件 p.20–22 · 按 Qwen 聊天模板格式重建（示意）

③ 一次工具调用在底层怎么发生

show_prompt.py — weather-demo

py show_prompt.py

```python
messages = [
  {"role": "system", "content": "Be concise."},
  {"role": "user", "content": "Weather in Pittsburgh?
]
weather_tool = {
  "type": "function",
  "function": {
    "name": "get_weather",
    "parameters": {
      "type": "object",
      "properties": {"city": {"type": "string"}},
      "required": ["city"]}}}
prompt = tok.apply_chat_template(
    messages, tools=[weather_tool], tokenize=False)
print(prompt)
```

zsh — 80×24

```
$ python show_prompt.py
<|im_start|>system
Be concise.<|im_end|>
<|im_start|>user
Weather in Pittsburgh?<|im_end|>

$ python show_prompt.py --tools
<|im_start|>system
Be concise.  # …（模板自带的工具说明，有删节）
<tools>
{"type":"function","function":{"name":"get_weather",…}}
</tools><|im_end|>
<|im_start|>user
Weather in Pittsburgh?<|im_end|>
<|im_start|>assistant
<tool_call>
{"name":"get_weather","arguments":{"city":"Pittsburgh"}}
</tool_call>
```

↑ 模型输出（示意）

API 返回时附上 id: "call_123"



课件 p.20–22 · 按 Qwen 聊天模板格式重建（示意）

同一个调用，各家写法不一样

同一份 schema：get_weather(city="Pittsburgh")

Qwen 3.8 函数 + 参数标签

```
<tool_call>
<function=get_weather>
<parameter=city>
Pittsburgh
</parameter>
</function>
</tool_call>
```

Mistral Small 4 控制 token + 调用 ID

```
[TOOL_CALLS] [{
  "name": "get_weather",
  "arguments":
    {"city": "Pittsburgh"},
  "id": "abc123xyz"
}]</s>
```

DeepSeek V3.2 DSML 块

```
<｜DSML｜function_calls>
 <｜DSML｜invoke
   name="get_weather">
 <｜DSML｜parameter name="city"
   string="true">Pittsburgh
 …</｜DSML｜function_calls>
```

Qwen 解析器

Mistral 解析器

DeepSeek 解析器

微调：沿用模型原来的格式

Hugging Face · apply_chat_template



课件 p.23

harness 怎么分发一次调用 以 OpenHands 为例

① 初始化时注册

工具说明
BashTool

→

解析配置
resolve_tool()

→

名字 → 定义
tools_map[name]

ToolDefinition
schema · 动作类型 · 执行器

加新工具 = 起个名字 + 挂一个函数

② 每次模型调用时分发

解析
{name, args}

→

查表
tools_map[name]

→

校验
action_from_arguments

→

执行
executor(action)

→

返回观察
ObservationEvent

× 调用本身不合法

× 真跑起来才报错

成功、失败 → 都回到模型



课件 p.24 · OpenHands 的分发流程

结果靠调用 ID 对回去

助手的工具调用
"id": "call_123"
"name": "get_weather"

工具结果
"tool_call_id": "call_123"
"content": "72°F · sunny"

模型

天气

日历

航班

回来的顺序是乱的：2 → 3 → 1，全靠 ID 归位



课件 p.25（中文重绘）

结果靠调用 ID 对回去

实践中常见：有的服务商（如 Anthropic）会判定历史不合法





课件 p.25（中文重绘）

约束解码：格式一定不出错

```
{"name": "get_weather", "arguments": {"city": "Pittsburgh"}
```

JSONDecodeError：少了一个 }

~~事后补括号~~

生成时就约束它 → 约束解码

只保证形式

语法
合法的 JSON

形状
必填字段都在

类型
city 是字符串

取值
units 只能是 C 或 F

内容是真的吗?

有权限吗?

工具选对了吗?

任务能完成吗?



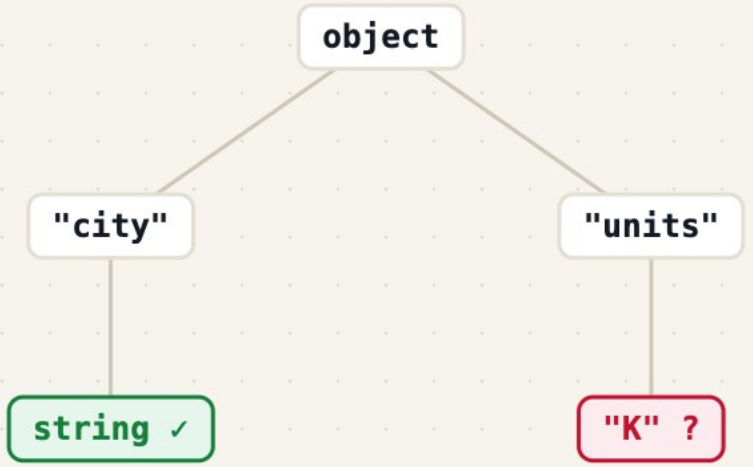
课件 p.27–28（中文重绘）

用 JSON Schema 写规则

```
{
  "type": "object",
  "properties": {
    "city":  {"type": "string"},
    "units": {"enum": ["C", "F"]}
  },
  "required": ["city"],
  "additionalProperties": false
}
```

`{"city":"Pgh","units":"F"}` ✓合法 JSON ✓合 schema

`{"city":"Pgh","units":"K"}` ✓合法 JSON ×开尔文不在允许的单位里





课件 p.29–32（中文重绘）

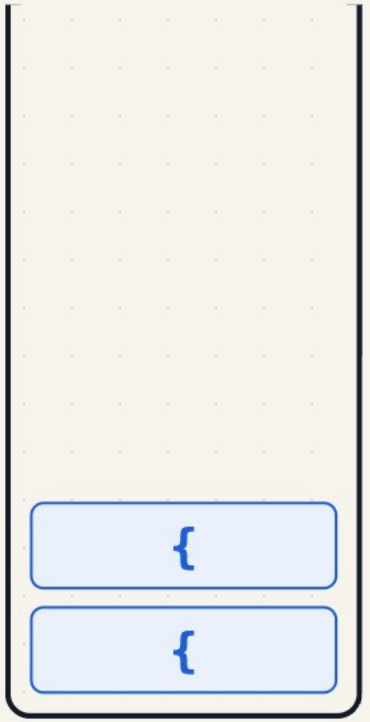
嵌套要记层数：下推自动机



有限状态机：每读一个符号，跳到下一个状态（正则那类规则够用）

JSON 能任意嵌套 {"a":{"b":{"c":...}}} → 状态机记不住层数

栈：嵌套到第几层、出来回到哪



"city":" → " c i t y " : "

一个 token 拆成字节，逐个喂进自动机



课件 p.29–32（中文重绘）

XGrammar：不合法的 token 直接设成 −∞

下一个 token 的候选和分数（示意）



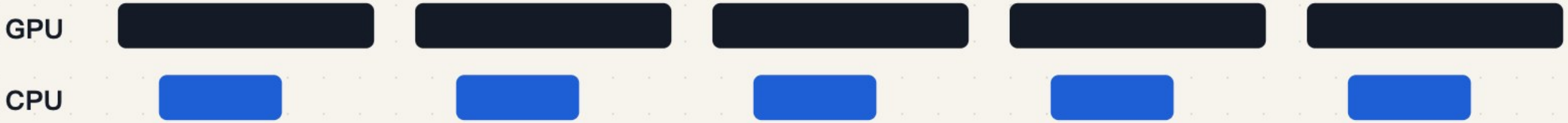
大部分 token

合不合法跟上下文无关 → 提前算好、缓存

少数 token

要看当前栈的状态 → 现场检查

CPU 上的检查，和 GPU 上的推理重叠进行





XGrammar（Dong et al., MLSys 2025）

什么情况下，还是不够?

```
{"name": "get_weather", "arguments": {"ci
```

输出 token 用完了：还走在合法的路上，但没走到终点



XGrammar（Dong et al., MLSys 2025）

最直接：给 agent 一个 Python 库

同一个进程

```python
# weather_lib.py
from typing import Literal
from pydantic import BaseModel
class Weather(BaseModel):
    summary: str
    temperature: float
def get_weather(
    city: str,
    units: Literal["C", "F"] = "F",
) -> Weather:
    return provider.lookup(city, units)
```

要从网络上调，还缺：

- 地址
- 协议
- 序列化
- 认证

库：带类型的编程接口，还不是网络服务



课件 p.37

用 FastAPI 把函数挂到网上

```python
# api.py
from fastapi import FastAPI, Security
from fastapi.security import APIKeyHeader
app = FastAPI(title="Weather API")
key_header = APIKeyHeader(name="X-API-Key")
@app.get("/weather", operation_id="get_weather")
def weather(city: str, key=Security(key_header),
            units: Literal["C", "F"] = "F"):
    verify(key)
    return get_weather(city, units)
```

openapi.json（FastAPI 自动生成）

地址	**GET /weather**
参数	`city: string`
取值范围	`units: enum ["C","F"]`
认证	**Header: X-API-Key**

→ 原样交给模型：它就知道能调哪些接口



课件 p.38–39

```
"city"
"units"
$ curl -sS -G https://weather.example/weather \
    --data-urlencode "city=Pittsburgh" \
    --data-urlencode "units=F" \
    -H "X-API-Key: $WEATHER_API_KEY"
{"summary": "sunny", "temperature": 72.0}

$ echo $WEATHER_API_KEY
wk_live_••••••••••••••••••••
```

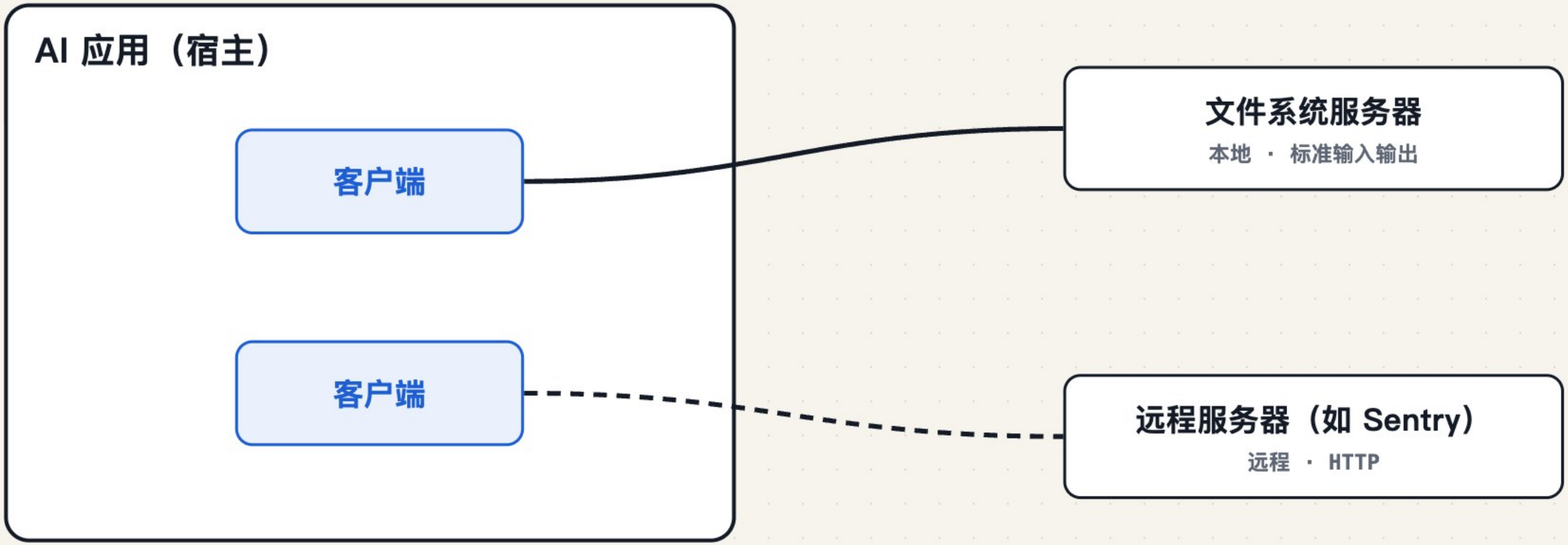
有 shell 权限 = 也拿到了这把 key



课件 p.40（终端为示意，key 已打码）

MCP：模型上下文协议

Anthropic 提出 · 给模型提供额外上下文的标准方式





课件 p.42–43（中文重绘）

为什么还要再跑一个程序?

很多人第一次见到 MCP 都会问

调接口已经有现成的好办法了，为什么还要 MCP?

MCP 是另一种面向 agent 的接口，不是调 REST API 的前提

FastAPI → 网页接口

FastMCP → MCP

openapi.json	
GET	/weather
GET	/forecast
POST	/alerts

get_weather　get_forecast　create_alert

FastMCP.from_openapi：每个接口自动变成一个 MCP 工具（接口名为示意）



课件 p.42–43（中文重绘）

MCP 真正的价值：凭证代管



账号被盗 · 所有仓库都可能被植入恶意代码



课件 p.44–45（中文重绘）

MCP 真正的价值：凭证代管

模型

两把 key，模型都看不到

agent
harness

MCP 服务器
校验 · 授权 · 分发

GitHub API

MCP key：只能登录这个服务器

上游 key（GitHub token）

给 agent 的东西，泄露了也无伤大雅；真正值钱的凭证，留在服务器那边

示例用静态 token，只适合教学和开发；生产环境校验 JWT 或用 OAuth



课件 p.44–45（中文重绘）

REST API 和 MCP：一张对照表

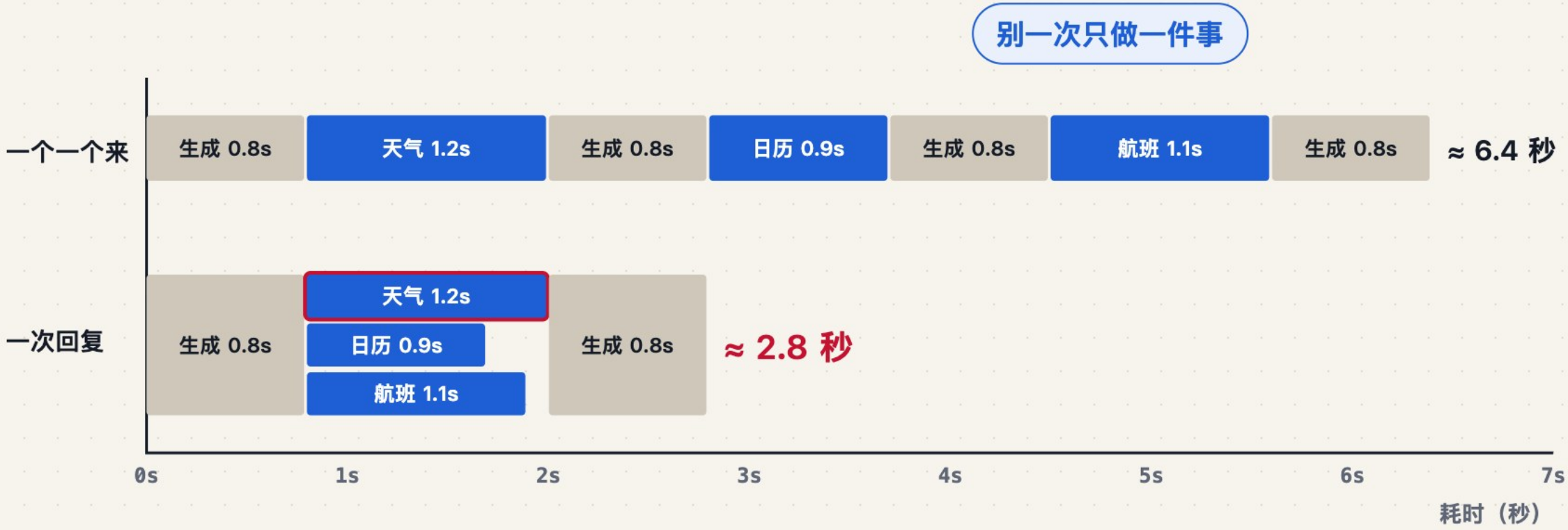
	OpenAPI / HTTP API	MCP
共同点	名字、描述，都用 JSON Schema 描述结构化输入	
发现能力	取一份 OpenAPI 文档	运行时调 `tools/list`
调用	HTTP 方法 + 地址 + 参数	通过 MCP 调 `tools/call`
认证	客户端直接向接口认证	客户端向 MCP 认证，上游认证另算
范围	只描述 HTTP 操作	还支持资源、提示词和扩展

OpenAPI 描述一个网页接口；MCP 规定 AI 应用怎么发现、调用各种能力 现成的：MCP Registry



课件 p.46

并行调用：一次往返做三件事



查客户 ID → 用 ID 取订单 → 给选中的订单退款　有依赖：只能按顺序



课件 p.48–49 的数字

并发执行：保留调用 ID

asyncio.gather

```python
async def execute(call):
    args = json.loads(call.arguments)
    value = await TOOLS[call.name](**args)
    return call.call_id, value
results = await asyncio.gather(
    *(execute(call) for call in function_calls),
    return_exceptions=True,
)
```

call_1 · 天气 72°F

call_2 · 日历 3 个会

call_3 · 航班 UA 812

一个失败，不拖垮其他的



课件 p.51

更贵的模型，按任务算可能更便宜

便宜的模型

单价低，但调用一长串、一次一件

总价 更高

更贵的模型

更快找到解法；几件事并排一起做



总价 更低

示意，不是真实账单

为什么最近半年才普及？强化学习：训练时对完成任务的耗时罚得很重 → 模型学会一次多发几个调用



课件 p.51

伯克利函数调用排行榜 BFCL · 第四版

单轮

简单 多个函数 并行 多函数并行

多轮

基础 缺函数 缺参数 长上下文

Agentic

网页搜索 记忆

稳健性

参数幻觉 格式敏感



课件 p.53–56（中文重绘）

评测要覆盖整个栈

层次	问的是	指标	要真执行吗	典型失败
选择	工具选对了吗	精确率 / 召回率	不用	漏调或多调
参数	值给对了吗	AST / schema 匹配	不用	字段或值错了
轨迹	顺序对吗	序列成功率	通常要	依赖关系错了
任务	目标达成了吗	端到端成功率	要	看着合理，其实错了



课件 p.53–56（中文重绘）

光答对还不够：运营指标和故障

效率

延迟 · 调用次数 · token · 成本

可靠性

超时 · 重试 · 部分失败

安全

权限 · 副作用

模型

选错工具　填错参数　无视结果

harness

解析　认证　校验　ID 对不上

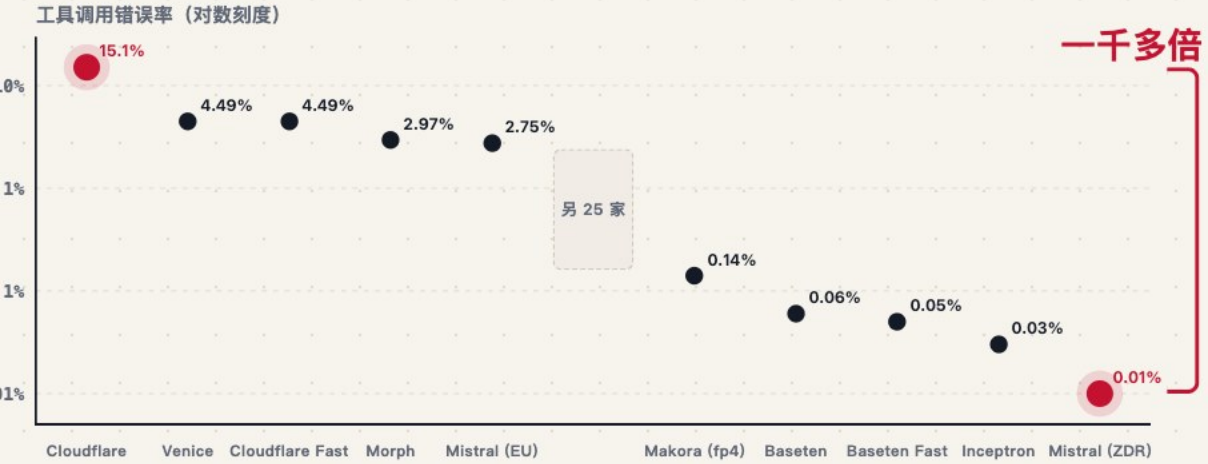
服务商和工具

超时　限流　执行出错



课件 p.53–56（中文重绘）

同一个模型，换个服务商：错误率差一千多倍



① 量化

FP8 基本是默认；压到 FP4，错误的调用明显变多

② 推测解码

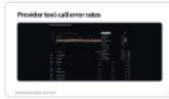
小模型先猜；有损的就可能影响结果

③ 约束解码没开

vLLM / SGLang 支持；没实现就全靠模型自己

④ 服务不稳定

时不时挂掉，调用自然失败



OpenRouter 服务商统计 · 2026-08-27（课件 p.57）

工具调用，是一个分层的系统

5	**系统**	编排依赖和并发；评测选择、参数、结果、成本和运营故障
4	**接口**	直接调 REST，或用 MCP：两种暴露能力、管理凭证的方式
3	**约束**	JSON Schema 和语法掩码保证形式，保证不了语义
2	**机制**	定义进提示词 → 按各家格式写出调用 → harness 校验、分发、按 ID 对回
1	**能力**	检索、执行、生成、应用里的动作：有了这些，agent 才是 agent



课件 p.59–60